

PROGRAMACIÓN II

TERCERA ENTREGA DEL TRABAJO PRÁCTICO

INTERFAZ GRÁFICA

Integrantes del grupo: Julián Acevedo y Sofía Díaz

Docentes: Ceceres, Adrian y Navarro Prieto, Joaquin

Comisión: com 5

Cuatrimestre - Año: 2c - 2025

CONCEPTOS UTILIZADOS EN EL TRABAJO

Uno de los requerimientos de esta tercera parte del trabajo, era utilizar conceptos aprendidos durante la cursada (StringBuilder, iteradores, foreach, herencia y polimorfismo, métodos abstractos, sobreescritura, sobrecarga e interfaces). A continuación, demostraremos dónde fueron utilizados dichos conceptos requeridos.

StringBuilder: utilizamos StringBuilder en los métodos toString() de las clases principales, como HomeSolution.java (entre las líneas 130 y 137) y Proyecto.java (entre las líneas 262 y 288).

Iteradores y foreach: ambos conceptos se utilizan más de una vez a lo largo del código. Podemos encontrarlos también en HomeSolution.java y Proyecto.java, donde son utilizados para recorrer listas y mapas. Por ejemplo, en HomeSolution.java, se utiliza en los métodos proyectosFinalizados(), empleados y tareasNoAsignadas. En Proyecto.java, se utiliza en los métodos tieneRetrasos() y calcularCosto().

Herencia: es utilizada en las clases EmpleadoContratado.java y EmpleadoPlanta.java. Dichas clases extienden (por eso utilizamos extends) la clase padre Empleado; esto permite que las dos subclases hereden atributos y métodos de Empleado.

Clases/métodos abstractos: son utilizadas en Empleado.java. La clase Empleado está declarada como abstracta (public abstract class Empleado).

Interfaces: utilizado en HomeSolution.java. La clase HomeSolution implementa la interfaz IHomeSolution (public class HomeSolution implements IHomeSolution) que nos fue dada.

Sobreescritura: utilizada en EmpleadoContratado.java y EmpleadoPlanta.java. Ambas clases sobrescriben el método calcularPago(double diasTrabajados) y el método toString().

Sobrecarga: utilizada en HomeSolution.java. Como se puede ver en el código, tenemos dos métodos con el mismo nombre (registrarEmpleado) pero con diferente número de parámetros. Uno de los métodos es registrarEmpleado(String nombre, double valor) y el otro es registrarEmpleado(String nombre, double valor, String categoria).

Polimorfismo: utilizado en HomeSolution.java y en Proyecto.java. Esto se puede ver, en Proyecto.java por ejemplo, cuando llamamos a e.calcularPago(t.obtenerDuracion()) dentro de calcularCosto() y se ejecuta la implementación correcta del método calcularPago para una clase específica de empleado (Planta o Contratado), sin importar que la referencia sea de Empleado.

Además de dichos conceptos, utilizamos ciertas estructuras de datos que fueron clave para el desarrollo del trabajo y explicaremos a continuación.

Primero, comenzaremos explicando las que elegimos usar en el archivo HomeSolution.Java. Allí podemos encontrar dos diccionarios:

- `Map<Integer, Empleado> empleados`: lo usamos para acceder rápido ($O(1)$) a un objeto `Empleado`, y utilizamos su legajo (por eso el tipo de dato es `Integer`, ya que el legajo de un empleado es `int`) como clave. Luego nos sirvió para métodos como `tieneRetrasos(Int legajo)` o `consultarEmpleado(Int legajo)`, donde su uso nos permitió realizar una búsqueda por ID de forma rápida y sin tener que recorrer toda una lista de empleados.
- `Map<Integer, Proyecto> proyectos`: igual que en el caso anterior, acá nos permite el acceso directo ($O(1)$) a un objeto `Proyecto` a través de su número (`Integer`). Lo usamos para operaciones como `consultarDomicilioProyecto(Integer número)` o `tareasDeUnProyecto(Integer numero)`.

Además de los mapas, utilizamos una lista:

- `List<Empleado> empleadosLibres`: se usa para mantener una colección de empleados que no tengan tareas asignadas. Elegimos utilizar una lista porque permite recorrer la lista de forma fácil y consideramos que, en este caso, el orden de inserción no era importante para la funcionalidad del trabajo.

También utilizamos dos listas en otra clase, la clase `Proyecto`:

- `List<Tarea> listaDeTareas`: contiene las tareas asociadas con un proyecto. Esta lista nos facilita el recorrido completo de las tareas para calcular el costo total en `calcularCosto()` o para buscar tareas sin asignar `tareasProyectoNoAsignadas()`.
- `List<Empleado> historialEmpleados`: registra a los empleados que participaron en el proyecto. La usamos para almacenar el historial de empleados y, en caso de que sea necesario, recorrer dicha historia.

IREP POR CADA TAD IMPLEMENTADO:

Empleado (clase abstracta):

- El legajo debe ser > 0
- El nombre no puede ser nulo ni vacío
- La cantidadRetrasos debe ser ≥ 0
- Si `ocupado = true`, el empleado está asignado a una tarea activa

EmpleadoPlanta:

- El `valorDia` debe ser > 0

- La categoria debe ser: "INICIAL", "TÉCNICO" o "EXPERTO"

EmpleadoContratado

- El valorHora debe ser > 0

Ciente:

- El nombre no puede ser nulo ni vacío

Proyecto:

- El numero del proyecto debe ser > 0
- listaDeTareas no puede ser nula ni vacía

Tarea:

- El titulo no puede ser nulo ni vacío (enforzado en el constructor)
- Los díasNecesarios deben ser > 0 (enforzado en el constructor)
- diasRetraso debe ser ≥ 0 (inicializado en 0)

Fecha:

- La combinación de dia/mes/año tiene que ser una fecha
- El mes debe estar entre 1 y 12
- El año debe ser ≥ 1

HomeSolution:

- Los mapas de empleados y proyectos no pueden ser nulos
- contadorProyectos y contadorLegajos tienen que ser ≥ 0

ANÁLISIS DE COMPLEJIDAD

Primero, analizaremos la complejidad de reasignarEmpleado:

```
public void reasignarEmpleado(Empleado nuevoEmpleado) {  
    if (this.empleado == null) { 1  
        throw new IllegalArgumentException("No existe un empleado asignado"); 1*1  
    }  
    if (!nuevoEmpleado.estaLibre()) { 1  
        throw new IllegalArgumentException("El empleado no está disponible"); 1*1  
    }  
}
```

```

    }

    this.empleado.liberar(); 1

    this.empleado = nuevoEmpleado; 1

    nuevoEmpleado.asignar(); 1

    }

```

$$F(n) = 1 + (1*1) + 1 + (1*1) + 1 + 1 + 1 =$$

$$F(n) = 7$$

El método reasignarEmpleado(Empleado nuevoEmpleado) tiene una complejidad constante ($O(1)$) ya que nos dio 7 como resultado y, según la regla 4 de álgebra de órdenes, $O(k) = O(1)$ para todo k constante. Por lo tanto:

$$F(n) = O(1)$$

Luego, vamos a analizar también la complejidad de reasignarEmpleadoConMenosRetraso:

```

public void reasignarEmpleadoConMenosRetraso(Integer número, String título) throws
Exception {

```

```

    Proyecto p = proyectos.get(numero); 1

    if (p == null) throw new Exception("Proyecto inexistente."); 1 + (1*1)

    if (p.estaFinalizado()) throw new Exception("Proyecto finalizado."); 1 + (1*1)

    if (empleadosLibres.isEmpty()) 1

        throw new Exception("No hay empleados disponibles."); 1*1

    Empleado mejor = null; 1

    for (Empleado e : empleadosLibres) { n (n = cantidad de empleados libres)

        if (mejor == null || e.getCantidadRetrasos() < mejor.getCantidadRetrasos()) { 1*n

            mejor = e; 1*n

        }

    }

    if (mejor == null) 1

        throw new Exception("No hay empleados disponibles."); 1*1

    p.reasignarEmpleado(título, mejor); 1 (queda 1 porque reasignarEmpleado es  $O(1)$ )

```

```
empleadosLibres.remove(mejor); 1
}
```

$$F(n) = 1 + 1 + (1*1) + 1 + (1*1) + 1 + (1*1) + 1 + n + (1*n) + (1*n) + 1 + (1*1) + 1 + 1$$

$$F(n) = 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + n + n + n + 1 + 1 + 1 + 1$$

$$F(n) = 11 + 3n$$

$$F(n) = O(n)$$

Por lo tanto:

$$F(n) = 11 + 3n$$

$$= O(11 + 3n)$$

$$= O(11) + O(3n) \quad (\text{regla 2})$$

$$= O(11) + O(3) * O(n) \quad (\text{regla 3})$$

$$= O(1) + O(1) * O(n) \quad (\text{regla 4})$$

$$= O(1) + O(n) \quad (\text{regla 3})$$

$$= O(\max\{1, n\}) \quad (\text{regla 2})$$

$$F(n) = O(n) \quad (\text{regla 1: } O(1) \leq O(\log n))$$